

yangzhou — 仓库指南(给 agent)

开源项目管理系统。核心:匹配引擎(Requirement × Capability → 单人输出可行性/技能差距,团队输出分配建议)。API-first:CLI 与 Web 都是 OpenAPI 契约的瘦客户端。栈:Kotlin + Spring Boot 4 / Postgres + JSONB / Next.js + MUI + TS / MIT。

真相源(动笔前按触发条件读)

- 命名实体或写用户可见文案前 → 词汇表 D:\obsidian\projects\yangzhou\CONTEXT.md (**Item** 不是 Task;Workspace/Project/Team/Member/Attribute/**Requirement/Capability**)
- 实现引擎、实体关系或输出形态前 → D:\obsidian\projects\yangzhou\DOMAIN.md (判定 5 形态、绿黄红聚合规则、领域规则 6 条)
- 实现某张票时 → Linear JCW-78~84(每票自带验收清单;父票 JCW-77 = spec 全文 docs/spec/0001-v1-core-and-matching.md)
- 动 **schema** 或架构前 → docs/adr/ (0001 三层容器 / 0002 Item 同质树 / 0003 统一属性 / 0004 monorepo 与工具链)

布局(ADR-0004)

```
backend/  Gradle 多模块:domain(纯 Kotlin,零 Spring)· api(REST 薄层)· persistence(Postgres)· cli
web/      Next.js + MUI + TS,瘦客户端
docs/     adr/ · spec/
```

领域硬规则

- domain 零 Spring/DB 依赖——引擎是纯函数,从 prototype 抬来(分支 prototype/matching-engine-output)。注意:这是独立 Gradle 模块,与 chess 单体应用"禁止 domain 包"的约定不同,属有意为之(ADR-0002/0003 的物理形态)。
- 角色永不进匹配输入(ADR-0003);等级刻度 1–4;presence 需求为默认,min-level 只用于关键路径。
- 用户可见文案中文优先,文案外置。

过程(chess 实战约定平移)

- 票即计划:实现前读 Linear 票 + spec 对应故事,不凭记忆。

- 外科手术式改动,不顺手重构无关代码。
- 最简可行实现,不加投机抽象。
- 非平凡行为变更必须带测试。
- 声称完成前跑通相关验证;后端以 `./gradlew build --no-daemon` 绿 + `bootRun` 能启动为准(临时端口验证,起后即停)。
- 大输出/多命令优先 `context-mode` 批处理工具,Bash 仅琐碎命令。
- `api` 模块内部按垂直切片组织(`project / item / attribute / ...`):实体+DTO+service+controller 同包;技术配置归 `config.*`。

Issue 工作流(PR 模式)

1. 开工: `git checkout main && git pull --ff-only && git checkout -b cwjiang/JCW-{N}-{short}`
2. 实现带测试,跑全相关验证
3. PR: `gh pr create --base main --title "JCW-{N}: ..."`
4. **停**,等用户审阅合并
5. 合并后同回合完成: `git checkout main && git pull --ff-only` → 删分支 → **Linear 移 Done**(非可选)→ 扫父票:子票全 Done 则父票 Done,否则 In Progress。PR 已合而 issue 停在 In Progress = 流程破损态。

Kotlin 风格

- 尽量 `val`;实体/DTO 用不可变 `data class`
- 更新用 `copy(...)` + `save`,用返回实例
- 小切片的 DTO/请求/响应放同一文件,吵了再拆

持久层(ADR-0004;chess 实战约定平移)

- Spring Data JDBC + Flyway;禁 JPA/Hibernate/Exposed/BaseEntity 继承
- 实体主构造直接声明 `id / objectId / createdAt / updatedAt`; `objectId` 在 Kotlin 生成
- 对外只暴露公共 ID(`projectId / itemId`),永不暴露内部数字 `id`;FK 用 `object_id` UUID 列
- migration 用 SQL `identity` + 命名约束;枚举 = Kotlin `enum` + `VARCHAR` + `CHECK` 约束
- 本机 Postgres 由共享 `compose` 提供(`F:/code/docker-compose/postgresql`),不建项目本地 `compose`;测试库 `yangzhou_test`

测试

- 唯一 seam = REST API 黑盒(spec);引擎纯函数直测。
- 单元测试:类级并行安全——无共享可变全局态、不碰真 DB/网络,不起 Spring;CI 跑。
- 集成测试: @SpringBootTest(RANDOM_PORT) + RestTestClient ;不用 MockMvc / @WebMvcTest / TestRestTemplate;不用 @Transactional 回滚——每测试自清自建、只造自己要的数据,断言响应体与库内状态。
- 敏感值(密码/token)永不进日志;登录失败消息保持笼统。

错误与契约

- service fail-fast 抛业务异常;HTTP 映射集中 GlobalExceptionHandler;统一错误形状 (code/message/path/可选 fields);请求校验用 Bean Validation
- OpenAPI + Scalar(不用 Swagger UI/Knife4j);注解最少(controller @Tag + 偶尔摘要),字段级只在生成文档误导时补
- 日志:默认 SLF4J/Logback,YAML 配级别;不加 MDC/关联 ID/自定义 logback 除非新决策
- 不加 mapstruct;slice 内手写小 mapper

构建与运行

(票 1 落地 Gradle 时补:JDK 25、 ./gradlew 命令、测试起法。)

工作流

main 干线开发;原型留 prototype/* 分支;每张票 = 一个 Linear issue,验收清单全绿才关票。